Frontend Architecture Guidelines

Team Engineering Standard | Topic 1: Architecture

Version 1.0 | Status: Proposed baseline | Applies to: React frontend applications

Purpose

A practical, team-facing baseline for React frontend architecture: folder structure, component design, routing, API layer, state management, and error handling.

1. Purpose and principles

This document defines a shared baseline for structuring and building frontend applications. The goal is to make the codebase consistent, maintainable, testable, secure, and easier for multiple developers to work on.

- Prefer clear responsibilities over large components that do everything.
- Keep related code together by feature where practical.
- Reuse shared components and utilities when behaviour is genuinely common.
- Keep server data, client UI state, and URL state conceptually separate.
- Make loading, empty, success, and error states explicit.
- Automate repeatable checks where practical; do not rely only on reviewer memory.

Note: *Project-specific decisions (framework versions, naming details, directory layout, deployment process) should be agreed by the Frontend Lead and documented alongside this standard.*

2. Recommended folder structure

For a React + Vite application, a feature-oriented structure is a useful starting point. Adapt it to the project; do not reorganize an established repository without an agreed migration plan.

```
src/
■■■ app/
■   ■■■ App.jsx
■   ■■■ router.jsx
■   ■■■ providers.jsx
■■■ components/
■   ■■■ ui/                # Generic shared UI: Button, Modal, Loader
■   ■■■ layout/            # Header, Footer, page shell
■■■ features/
■   ■■■ products/
■   ■   ■■■ pages/
■   ■   ■   ■■■ ProductsPage.jsx
■   ■   ■■■ components/
■   ■   ■   ■■■ ProductCard.jsx
■   ■   ■   ■■■ ProductFilters.jsx
■   ■   ■■■ hooks/
■   ■   ■   ■■■ useProducts.js
■   ■   ■■■ services/
■   ■   ■   ■■■ productService.js
■   ■   ■■■ utils/
■   ■■■ cart/
■       ■■■ components/
■       ■■■ hooks/
■       ■■■ context/
■■■ lib/
■   ■■■ api-client.js
■■■ hooks/                 # Truly cross-feature hooks only
■■■ utils/                 # Truly cross-feature utilities only
■■■ assets/
■■■ styles/
■■■ main.jsx
```

- app/: application composition, routing, and top-level providers.
- components/ui/: generic reusable components without feature-specific business logic.
- components/layout/: shared page structure such as navigation and footer.
- features/: feature-specific pages, components, hooks, services, and helpers.
- lib/: configured third-party clients and infrastructure, such as Axios.
- hooks/ and utils/: use only for code shared across multiple features; otherwise keep code with its feature.

Note: *Avoid creating empty folders or excessive layers just to follow a diagram. Use the smallest structure that makes ownership and dependencies clear.*

3. Component design

Components should have clear responsibilities and predictable interfaces. Split components when it improves readability, reuse, testing, or independent development—not merely to reduce line count.

Component categories

- Page components: represent routes and coordinate feature-level UI.
- Feature components: implement domain-specific UI, such as ProductCard or CartItem.
- Shared UI components: generic elements such as Button, Input, Modal, and Loader.
- Layout components: shared structure such as Header, Sidebar, and Footer.

Component rules

- Prefer props for parent-to-child data and callback props for child-to-parent events.
- Keep local state local unless multiple components need to share it.
- Use stable keys when rendering lists.
- Use semantic HTML and accessible names for interactive elements.
- Keep API transport details out of presentational components where practical.
- Do not introduce abstractions until there is a clear reuse or maintainability benefit.

```
function ProductCard({ product, onAddToCart }) {
  return (
    <article>
      <img src={product.image} alt={product.title} />
      <h2>{product.title}</h2>
      <p>■{product.price}</p>
      <button onClick={() => onAddToCart(product)}>
        Add to Cart
      </button>
    </article>
  );
}
```

In this example, ProductCard renders a product and reports the user's action through a callback. It does not need to know how products are fetched or how the entire cart is persisted.

4. Routing

Routing maps URLs to page components and controls navigation. Define routes centrally (or in a clearly owned route module) so route behaviour is easy to discover.

- Use meaningful, stable paths, such as /products, /products/:productId, /cart, and /login.
- Use route parameters for resource identity and query parameters for shareable page state such as search, sort, filters, and pagination when appropriate.
- Use protected route patterns for authenticated areas, but always enforce authorization on the server as well.
- Provide a not-found route and a clear strategy for route-level loading and errors.
- Use lazy loading for suitable route-level chunks when it benefits performance.
- Do not rely on hiding a UI route as a security boundary.

```
const routes = [
  { path: "/products", element: <ProductsPage /> },
  { path: "/products/:productId", element: <ProductDetailsPage /> },
  { path: "/cart", element: <CartPage /> },
  { path: "/login", element: <LoginPage /> },
];
```

Note: *The snippet is illustrative; use the routing library and configuration already adopted by the project (for example, React Router).*

5. API layer

Keep network configuration and endpoint operations out of components. Use a shared API client for common transport settings and feature services for endpoint-specific calls.

Recommended responsibility flow

```
Component/Page → Query hook or feature hook → Feature service → Shared API client → Backend API
```

- API client: base URL, common headers, timeout, and shared request/response handling.
- Feature service: endpoint paths, request parameters, and response mapping for one feature.
- Query hook: data fetching lifecycle, caching, refetching, and server-state integration.
- Component: displays data and responds to user actions; it should not duplicate low-level request configuration.
- Read environment-specific configuration from environment variables; Vite variables prefixed with VITE_ are exposed to client code and must never contain secrets.
- Handle non-success responses consistently and avoid exposing sensitive server details to users.

```
// lib/api-client.js
import axios from "axios";

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10000,
});

export default apiClient;

// features/products/services/productService.js
import apiClient from "../../../lib/api-client";

export async function getProducts() {
  const response = await apiClient.get("/products");
  return response.data;
}
```

Note: *If the project uses TanStack Query, keep query keys consistent and include every input that changes the returned data (for example category or page). Avoid duplicating server data in Context or Redux without a clear reason.*

6. State management

Choose state management based on the type of state, not by defaulting every value to a global store.

State type	Examples	Typical approach
Local UI state	Modal open, input value	useState / useReducer
Shared client state	Cart, theme, UI preferences	Context or Redux Toolkit when complexity warrants it
Server state	Products, orders, profile from API	TanStack Query or the project's chosen data-fetching solution
URL state	Search, filter, sort, page	Route params and query string when shareable/bookmarkable
Persistent browser state	Non-sensitive preferences	localStorage or sessionStorage as appropriate

- Do not store the same source of truth in multiple places unless synchronization is intentionally designed.
- Do not store passwords, access tokens, or other secrets in ordinary browser storage. Follow the organization's authentication design; HttpOnly cookies can reduce JavaScript access to session credentials.
- Use Context for genuinely shared values; avoid turning one context into a store for every unrelated state.
- Use Redux Toolkit when the application benefits from explicit centralized updates, tooling, or complex shared client state—not merely because the app has many API calls.

7. Error handling and UI states

Every API-driven screen should have a deliberate approach to pending, success, empty, and failure states. Users should receive helpful feedback without seeing internal stack traces or sensitive details.

- Pending: show a loader or skeleton when waiting is meaningful.
- Success: render the returned data.
- Empty: explain that no results are available and provide a next step when useful.

- Failure: show a clear message and retry action when retrying is safe.
- Form validation: explain what needs to be corrected and associate messages with fields.
- Unexpected render errors: use an error boundary where appropriate so one render failure does not blank the entire application.
- Logging: send diagnostic details to the approved monitoring system; do not expose secrets or personal data in logs.
- Authentication/authorization: handle 401/403 responses consistently and avoid infinite refresh/retry loops.

```
const { data, isPending, isError, refetch } = useProducts();

if (isPending) return <Loader />;
if (isError) {
  return <ErrorMessage onRetry={() => refetch()} />;
}
if (!data?.length) return <EmptyState message="No products found." />;

return <ProductList products={data} />;
```

Note: *The exact status flags and retry APIs depend on the data-fetching library and version. Do not blindly retry non-idempotent operations such as placing an order.*

8. Quality gates and ownership

Architecture standards are most effective when the team can verify them. Use code review for design decisions and automation for repeatable checks.

- ESLint: enforce applicable correctness and code-quality rules.
- Prettier: format files consistently.
- TypeScript, if adopted: run a type check in CI.
- Tests: cover critical component behaviour and important user flows.
- Accessibility: use semantic HTML, keyboard checks, and automated scans as appropriate.
- CI: run lint, tests, type checking (if applicable), and production build on pull requests.
- Repository rules: require relevant status checks and approvals before merging into protected branches.

Responsibilities

- Frontend Lead: owns the baseline, approves significant architecture changes, and resolves cross-feature disagreements.
- Feature developer: follows the structure, tests the implementation, and documents deviations.
- Reviewer: checks correctness, maintainability, security, accessibility, and consistency with the agreed architecture.
- Team: proposes improvements and updates this document when decisions change.

Definition of done — architecture

- Code lives in the agreed feature or shared location.
- Component responsibilities and data flow are clear.
- Routing, API calls, and state follow the project's agreed patterns.
- Loading, empty, and error states are addressed where relevant.
- Sensitive values are not committed or exposed in client-side configuration.
- Required lint, tests, type checks (if applicable), and build checks pass.
- Any intentional deviation from this standard is explained in the PR.

9. Review and change control

This document is a proposed baseline. The Frontend Lead and team should confirm decisions that depend on the existing codebase, repository permissions, backend contracts, security requirements, and deployment process.

Suggested repository location: docs/frontend-architecture-guidelines.md. Keep the document versioned with the code and update it through a PR when the team's architecture decisions change.